LEMBAR KERJA PRAKTIKUM DOCKER

Topik **:** Kontainerisasi Aplikasi Web & Orchestration Multi-Container

Waktu **:** 90 Menit

A. Skenario Studi Kasus

Sebuah perusahaan rintisan sedang mengembangkan aplikasi **Katalog Produk berbasis REST API**. Aplikasi ini menggunakan **Node.js (Express)** untuk layanan web dan **PostgreSQL** sebagai database utama.

Saat ini tim developer mengalami kendala kompatibilitas lingkungan (*"works on my machine"*). Anda diminta oleh tim DevOps untuk melakukan kontainerisasi pada aplikasi tersebut menggunakan Docker dan mengorkestrasinya dengan Docker Compose agar sistem terisolasi, aman, dan mudah di-deploy.

B. Soal & Tugas Praktikum

Soal 1: Pembuatan & Optimalisasi Dockerfile

Buatlah berkas `Dockerfile` untuk aplikasi web Node.js dengan kriteria berikut:

- Menggunakan *base image* `node:20-alpine` untuk efisiensi ukuran.
- Mengatur *working directory* di `/app`.
- Menerapkan *layer caching* dengan menyalin berkas `package.json` terlebih dahulu sebelum menyalin seluruh kode sumber.
- Mengisolasi hak akses eksekusi dengan menjalankan aplikasi menggunakan user non-root (`node`).
- Membuka (*expose*) port `3000`.

Soal 2: Orchestration dengan Docker Compose

Buatlah berkas `docker-compose.yml` untuk menghubungkan aplikasi web dan database PostgreSQL dengan spesifikasi:

- **Service `web`:**
  - Di-build secara lokal dari `Dockerfile` di Soal 1.
  - Mapping port host `8080` ke port container `3000`.
  - Bergantung pada service `db` (`depends_on`) dan hanya berjalan setelah database dalam status *healthy*.
- **Service `db`:**
  - Menggunakan *image* `postgres:16-alpine`.
  - Menentukan *environment variables* untuk `POSTGRES_USER`, `POSTGRES_PASSWORD`, dan `POSTGRES_DB`.
  - Memiliki mekanisme *healthcheck* (menggunakan `pg_isready`).
- **Storage & Jaringan:**
  - Menggunakan *named volume* (`db_data`) agar data PostgreSQL tidak hilang saat container dihentikan/dihapus.
  - Menggunakan *custom bridge network* bernama `app_net` untuk mengisolasi komunikasi antar-container.

Soal 3: Eksekusi CLI & Pengujian System

Tuliskan perintah Docker CLI secara berurutan pada laporan untuk:

1. Menjalankan seluruh *stack* aplikasi di latar belakang (*detached mode*).
2. Memeriksa status log container `web` untuk verifikasi koneksi database.
3. Mengakses *shell* di dalam container `db` untuk mengecek daftar database.
4. Menghentikan dan menghapus seluruh container serta network tanpa menghapus *volume*.

C. Kriteria & Rubrik Penilaian

Indikator Penilaian	Deskripsi Kriteria	Bobot
Dockerfile Optimization	* Mengaplikasikan `node:20-alpine` * Bekerja pada `/app` & memanfaatkan *layer caching* * Menggunakan user non-root (`node`) demi keamanan	**30%**
Docker Compose Orchestration	* Konfigurasi port forwarding (`8080:3000`) & environment variables tepat * Penerapan *healthcheck* & *dependency condition* benar * Persistensi data dengan *named volume* & isolasi *custom network* berfungsi	**45%**
Pengujian & Perintah CLI	* Ketepatan urutan perintah Docker CLI	**25%**

D. Format Pengumpulan Output

1. Berkas `Dockerfile`
2. Berkas `docker-compose.yml`
3. Laporan ringkas (PDF) berisi *screenshot* bukti pengujian aplikasi di browser/Postman beserta *log output* terminal.